### 8.2.2 Making a spreadsheet with divs

Why use  elements? Because we're going to start moving our cells and rows around however we want, and by the time we override all the styles that make a table and its constituent elements work, we're better off starting fresh with a div. By setting the  position to absolute, we can use D3 transitions to move them around in the same way we moved SVG around in our earlier examples. We need to apply a bit more CSS to make the  elements take up the right amount of space, whereas *<table>* does that for us, but the added flexibility is worth it. A quick note for those of you who, like me, always forget the one crazy rule of positioning DOM elements: elements set to `position:relative` need to have a parent set to `position:relative` or `position:absolute`. We'll create a parent  (`div.table`) with `position:relative` to hold the  elements that make up our table.

**Listing 8.4 A spreadsheet made of divs**

```
d3.json("tweets.json",function(error,data) {
     createSpreadsheet(data.tweets)});

      function createSpreadsheet(incData) {

        var keyValues = d3.keys(incData[0]);

        d3.select("#traditional")
        .append("div")
        .attr("class", "table");

        d3.select("div.table")
        .append("div")
        .attr("class", "head")
        .selectAll("div.data")
        .data(keyValues)
        .enter()
        .append("div")
        .attr("class", "data")
        .html(function (d) {return d})
        .style("left", function(d,i) {return (i * 100) + "px";});

        d3.select("div.table")
        .selectAll("div.datarow")
        .data(incData, function(d) {return d.content})
        .enter()
        .append("div")
        .attr("class", "datarow")
        .style("top", function(d,i) {return (40 + (i * 40)) + "px";});

        d3.selectAll("div.datarow")
        .selectAll("div.data")
        .data(function(d) {return d3.entries(d)})
        .enter()
        .append("div")
        .attr("class", "data")
        .html(function (d) {return d.value})
        .style("left", function(d,i,j) {return (i * 100) + "px";});
   };
```

**It's a <div.table>, not a <table>.**

**Same as before**

**Instead of x/y or transform, HTML elements have top/bottom/left/right**